The CorrCal Likelihood Gradient on the GPU

Algebra of `grad_nll_kernels.cu`, kernel by kernel

Notes for `gpu_corrcal`

October 2026

Abstract

These notes derive the gradient of the CorrCal negative log-likelihood with respect to the antenna gains and follow it through the four GPU kernels that evaluate it. The derivation reproduces Pascua et al. (2025), Sec. 4.2.2. The new content is the factorization the kernels use. Two inner products that CPU CorrCal evaluates directly, $\mathbf{\Sigma}_g^T \mathbf{C}^{-1} \boldsymbol{d}$ and $\mathbf{\Delta}_g^T \mathbf{C}^{-1} \boldsymbol{d}$, are recast as Wiener-filter estimates of the source and diffuse mode amplitudes, computed from the Woodbury factors that the inverse already produced. In float32 this recasting is the difference between a correct gradient and one with no correct digits in its source projection.

Contents

1 The model and the objects in memory **2**
- 1.1 Data layout . . . 2
- 1.2 Gains as a real block-diagonal matrix . . . 2
- 1.3 Covariance . . . 2
- 1.4 What the inverse leaves behind . . . 2

2 The gradient **3**
- 2.1 Differentiating the likelihood . . . 3
- 2.2 The χ^2 term and the auxiliary vectors $\boldsymbol{q}$, s, t . . . 3
- 2.3 The trace term and the inverse power $\bar{P}_k$ . . . 4
- 2.4 Chain rule to the antenna gains . . . 4
- 2.5 The phase prior and the gain scale . . . 5
- 2.6 The complete gradient . . . 5

3 The vector q: two projections and a precision trap **5**
- 3.1 Structure . . . 5
- 3.2 What the projections mean . . . 6
- 3.3 Why the direct sum fails in float32 . . . 6
- 3.4 Computing the projections without cancellation . . . 6

4 The kernels **7**
- 4.1 K1a: the source estimate's right-hand side . . . 8
- 4.2 K1b: the two Wiener estimates . . . 8
- 4.3 K2: everything that happens on one baseline . . . 9
- 4.4 K3: from baselines to antennas . . . 9

5 Summary 10

1 The model and the objects in memory

1.1 Data layout

A visibility V_k on baseline k is complex. The pipeline stores every complex quantity as two real rows: row $2k$ holds the real part and row $2k+1$ the imaginary part. With N_{bl} baselines the data vector is

$$\boldsymbol{d} \in \mathbb{R}^{n_{\text{row}}}, \qquad n_{\text{row}} = 2N_{\text{bl}}, \tag{1}$$

sorted so that baselines in the same quasi-redundant group are contiguous. Group b occupies rows $e_b \le r < e_{b+1}$, where e_b is `edges[b]`. All edges are even, so a group never splits a Re/Im pair.

The gains are N_{ant} complex numbers $g_a = x_a + iy_a$, stored as `gains[2a]` $= x_a$ and `gains[2a+1]` $= y_a$. Baseline k joins antennas $k_1 =$ `ant_1[k]` and $k_2 =$ `ant_2[k]`, and its gain product is

$$G_k = g_{k_1}\, g_{k_2}^{*}, \qquad G_k^R = x_1x_2 + y_1y_2, \qquad G_k^I = y_1x_2 - x_1y_2, \tag{2}$$

where subscripts 1, 2 abbreviate k_1, k_2.

1.2 Gains as a real block-diagonal matrix

Multiplying a complex number $z = z^R + iz^I$ by G_k acts on the pair (z^R, z^I) as the 2×2 matrix

$$\mathbf{G}_k = \begin{pmatrix} G_k^R & -G_k^I \\ G_k^I & G_k^R \end{pmatrix} = G_k^R\, \mathbf{I}_2 + G_k^I\, \mathbf{J}, \qquad \mathbf{J} = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}. \tag{3}$$

The full gain matrix $\mathbf{G}$ is block diagonal with blocks $\mathbf{G}_k$. This is exactly what `apply_gains.cu` applies to every column of a matrix.

1.3 Covariance

The sky model has a diffuse part $\mathbf{\Delta}$ (block diagonal over redundant groups, n_{eig} columns per group) and a point-source part $\mathbf{\Sigma}$ (dense, n_{src} columns). With diagonal thermal noise $\mathbf{N}$, the covariance of the gain-corrupted data is

$$\mathbf{C} = \mathbf{N} + \mathbf{G}\,\mathbf{S}\,\mathbf{G}^T, \qquad \mathbf{S} = \mathbf{\Delta}\mathbf{\Delta}^T + \mathbf{\Sigma}\mathbf{\Sigma}^T. \tag{4}$$

Gains do not touch the noise. Writing the gained factors as

$$\mathbf{\Delta}_g = \mathbf{G}\mathbf{\Delta}, \qquad \mathbf{\Sigma}_g = \mathbf{G}\mathbf{\Sigma}, \tag{5}$$

gives $\mathbf{C} = \mathbf{N} + \mathbf{\Delta}_g\mathbf{\Delta}_g^T + \mathbf{\Sigma}_g\mathbf{\Sigma}_g^T$, which is the form the inverse operates on.

1.4 What the inverse leaves behind

The two-level Woodbury inverse proceeds as follows.

Diffuse level. Define the block-diagonal matrix $\mathbf{M} = \mathbf{N} + \mathbf{\Delta}_g\mathbf{\Delta}_g^T$. For each group b,

$$\mathbf{T}_b = \mathbf{\Delta}_{g,b}^T\mathbf{N}_b^{-1}\mathbf{\Delta}_{g,b}, \qquad \mathbf{L}_b\mathbf{L}_b^T = \mathbf{I} + \mathbf{T}_b, \qquad \bar{\mathbf{\Delta}}_b = \mathbf{N}_b^{-1}\mathbf{\Delta}_{g,b}\,\mathbf{L}_b^{-T}, \tag{6}$$

so that $\mathbf{M}^{-1} = \mathbf{N}^{-1} - \bar{\mathbf{\Delta}}\bar{\mathbf{\Delta}}^T$.

Source level. The source columns are then whitened against $\mathbf{M}$:

$$\mathbf{B}_b = \mathbf{\Sigma}_{g,b}^T \bar{\mathbf{\Delta}}_b, \qquad \mathbf{\Lambda} = \mathbf{\Sigma}_g^T \mathbf{M}^{-1} \mathbf{\Sigma}_g = \sum_b \left(\mathbf{\Sigma}_{g,b}^T \mathbf{N}_b^{-1} \mathbf{\Sigma}_{g,b} - \mathbf{B}_b \mathbf{B}_b^T \right), \tag{7}$$

$$\mathbf{L}_s \mathbf{L}_s^T = \mathbf{I} + \mathbf{\Lambda}, \qquad \bar{\mathbf{\Sigma}} = \mathbf{M}^{-1} \mathbf{\Sigma}_g \, \mathbf{L}_s^{-T}, \tag{8}$$

and the full inverse is

$$\mathbf{C}^{-1} = \mathbf{N}^{-1} - \bar{\mathbf{\Delta}} \bar{\mathbf{\Delta}}^T - \bar{\mathbf{\Sigma}} \bar{\mathbf{\Sigma}}^T. \tag{9}$$

Contents of VRAM. After `apply_gains`, `inv()` and one matvec, the device holds everything in Table 1. The gradient allocates nothing of size n_{row} beyond these.

Symbol	Shape	Where it lives
$\mathbf{\Delta}$, $\mathbf{\Sigma}$, $\mathbf{N}$	$(n_{\text{row}}, n_{\text{eig}})$, $(n_{\text{row}}, n_{\text{src}})$, (n_{row})	`cov.diff_mat`, `cov.src_mat`, `cov.noise`
$\mathbf{\Delta}_g$, $\mathbf{\Sigma}_g$	as above	`ws.diff_g`, `ws.src_g`
$\bar{\mathbf{\Delta}}$, $\bar{\mathbf{\Sigma}}$	as above	`ws.inv.diffuse_bar`, `ws.inv.sig_prime`
$\mathbf{R}_b = \mathbf{L}_b^{-T}$	$(N_{\text{grp}}, n_{\text{eig}}, n_{\text{eig}})$	`ws.inv.L_del` (upper triangular)
$\mathbf{B}_b$	$(N_{\text{grp}}, n_{\text{src}}, n_{\text{eig}})$	`ws.inv.B`
$\mathbf{L}_s^{-1}$	$(n_{\text{src}}, n_{\text{src}})$	`ws.inv.L_sig_inv` (lower triangular)
$\boldsymbol{p} = \mathbf{C}^{-1}\boldsymbol{d}$	(n_{row})	output of `cinv @ data`

Table 1: Inputs to the gradient launcher. Everything is float32.

2 The gradient

2.1 Differentiating the likelihood

Up to a constant, CorrCal minimizes

$$\mathcal{L} = \log\det \mathbf{C} + \boldsymbol{d}^T \mathbf{C}^{-1} \boldsymbol{d} + \frac{1}{N_{\text{ant}}^2 \sigma_\phi^2} \Big(\sum_a \phi_a \Big)^2, \qquad \phi_a = \text{atan2}(y_a, x_a). \tag{10}$$

Let ∂ denote the derivative with respect to any one real gain parameter. Using $\partial \log\det \mathbf{C} = \text{Tr}(\mathbf{C}^{-1}\partial\mathbf{C})$ and $\partial\mathbf{C}^{-1} = -\mathbf{C}^{-1}\partial\mathbf{C}\,\mathbf{C}^{-1}$,

$$\partial\mathcal{L} = \underbrace{\text{Tr}\left(\mathbf{C}^{-1}\partial\mathbf{C}\right)}_{\text{trace term}} \underbrace{- \boldsymbol{p}^T \partial\mathbf{C}\, \boldsymbol{p}}_{\chi^2 \text{ term}} + \partial\mathcal{L}_\phi. \tag{11}$$

Only $\mathbf{G}$ depends on the gains, so

$$\partial\mathbf{C} = \partial\mathbf{G}\,\mathbf{S}\,\mathbf{G}^T + \mathbf{G}\,\mathbf{S}\,\partial\mathbf{G}^T. \tag{12}$$

2.2 The χ^2 term and the auxiliary vectors q, s, t

Insert (12). The two pieces are transposes of one another, so

$$\boldsymbol{p}^T \partial\mathbf{C}\, \boldsymbol{p} = 2\, \boldsymbol{p}^T \partial\mathbf{G}\, \boldsymbol{q}, \qquad \boxed{\boldsymbol{q} = \mathbf{S}\,\mathbf{G}^T \boldsymbol{p}} \tag{13}$$

The vector $\boldsymbol{q}$ is the ungained sky covariance applied to the gain-uncorrupted residual $\mathbf{G}^T\boldsymbol{p}$. Since $\partial\mathbf{G}$ is block diagonal, the quadratic form splits into baselines. On baseline k, with $\boldsymbol{p}_k = (p_k^R, p_k^I)$, $\boldsymbol{q}_k = (q_k^R, q_k^I)$ and $\partial\mathbf{G}_k = \partial G_k^R\,\mathbf{I}_2 + \partial G_k^I\,\mathbf{J}$,

$$\boldsymbol{p}_k^T \partial\mathbf{G}_k \boldsymbol{q}_k = s_k\,\partial G_k^R + t_k\,\partial G_k^I, \qquad \begin{aligned} s_k &= p_k^R q_k^R + p_k^I q_k^I = \mathrm{Re}(p_k q_k^*), \\ t_k &= p_k^I q_k^R - p_k^R q_k^I = \mathrm{Im}(p_k q_k^*). \end{aligned} \tag{14}$$

The χ^2 term is therefore

$$-\boldsymbol{p}^T \partial\mathbf{C}\,\boldsymbol{p} = -2\sum_k \left(s_k\,\partial G_k^R + t_k\,\partial G_k^I\right). \tag{15}$$

2.3 The trace term and the inverse power $\bar{P}_k$

Write $\partial\mathbf{G} = \mathbf{E}\,\mathbf{G}$ with $\mathbf{E} = \partial\mathbf{G}\,\mathbf{G}^{-1}$. Each block $\mathbf{E}_k$ represents multiplication by the complex number $\partial G_k / G_k$, so $\mathbf{E}_k = \alpha_k\mathbf{I}_2 + \beta_k\mathbf{J}$ with

$$\alpha_k = \mathrm{Re}\,\frac{\partial G_k}{G_k} = \frac{G_k^R\,\partial G_k^R + G_k^I\,\partial G_k^I}{|G_k|^2}. \tag{16}$$

Then $\partial\mathbf{C} = \mathbf{E}(\mathbf{C}-\mathbf{N}) + (\mathbf{C}-\mathbf{N})\mathbf{E}^T$. Cyclic invariance of the trace gives

$$\mathrm{Tr}(\mathbf{C}^{-1}\partial\mathbf{C}) = 2\,\mathrm{Tr}\left((\mathbf{C}-\mathbf{N})\mathbf{C}^{-1}\mathbf{E}\right). \tag{17}$$

By (9), $(\mathbf{C}-\mathbf{N})\mathbf{C}^{-1} = \mathbf{I} - \mathbf{N}\mathbf{C}^{-1} = \mathbf{N}\mathbf{W}$ with $\mathbf{W} = \bar{\boldsymbol{\Delta}}\bar{\boldsymbol{\Delta}}^T + \bar{\boldsymbol{\Sigma}}\bar{\boldsymbol{\Sigma}}^T$. Both $\mathbf{N}$ and $\mathbf{E}$ are block diagonal, so only the 2×2 diagonal blocks $\mathbf{W}_k$ of $\mathbf{W}$ survive:

$$\mathrm{Tr}(\mathbf{C}^{-1}\partial\mathbf{C}) = 2\sum_k \mathrm{Tr}\left(\mathbf{W}_k\,\mathbf{E}_k\mathbf{N}_k\right). \tag{18}$$

If the Re and Im rows share a noise variance, $\mathbf{N}_k = \sigma_k^2\mathbf{I}_2$, then $\mathrm{Tr}(\mathbf{W}_k\mathbf{J}) = 0$ because $\mathbf{W}_k$ is symmetric. What remains is

$$\mathrm{Tr}(\mathbf{C}^{-1}\partial\mathbf{C}) = 2\sum_k \frac{\sigma_k^2\,\bar{P}_k}{|G_k|^2}\left(G_k^R\,\partial G_k^R + G_k^I\,\partial G_k^I\right), \tag{19}$$

$$\bar{P}_k = \mathrm{Tr}\,\mathbf{W}_k = \sum_e \left(\bar{\Delta}_{2k,e}^2 + \bar{\Delta}_{2k+1,e}^2\right) + \sum_c \left(\bar{\Sigma}_{2k,c}^2 + \bar{\Sigma}_{2k+1,c}^2\right). \tag{20}$$

This is Pascua et al. Eq. 89. The quantity $\bar{P}_k$ is the power that the low-rank part of $\mathbf{C}^{-1}$ places on baseline k. It is a row-local sum of squares and needs no reduction.

Remark. *CorrCal reads only* `noise[2k]`*, so it takes* $\sigma_k^2 = N_{2k}$*. With unequal Re/Im noise* (n_0, n_1) *the exact block trace is* $\alpha_k(n_0W_{00} + n_1W_{11}) + \beta_k W_{01}(n_0 - n_1)$*, and* (19) *becomes an approximation. The GPU code mirrors CorrCal exactly. Whether the exact form is preferable is the noise question still open from the Hessian notes.*

2.4 Chain rule to the antenna gains

From (2), the partial derivatives of (G_k^R, G_k^I) with respect to the four real gain parameters on baseline k are

$$\frac{\partial(G^R, G^I)}{\partial x_1} = (x_2, -y_2), \quad \frac{\partial(G^R, G^I)}{\partial y_1} = (y_2, x_2), \quad \frac{\partial(G^R, G^I)}{\partial x_2} = (x_1, y_1), \quad \frac{\partial(G^R, G^I)}{\partial y_2} = (y_1, -x_1). \tag{21}$$

Substituting into (15) and (19), with prefactor $f_k = 2\sigma_k^2 \bar{P}_k / |G_k|^2$, gives four numbers per baseline:

$$\begin{aligned}
c_k^{(1,R)} &= -2\,(x_2 s_k - y_2 t_k) + f_k\,(G^R x_2 - G^I y_2) \quad \to\ \partial\mathcal{L}/\partial x_{k_1},\\
c_k^{(1,I)} &= -2\,(y_2 s_k + x_2 t_k) + f_k\,(G^R y_2 + G^I x_2) \quad \to\ \partial\mathcal{L}/\partial y_{k_1},\\
c_k^{(2,R)} &= -2\,(x_1 s_k + y_1 t_k) + f_k\,(G^R x_1 + G^I y_1) \quad \to\ \partial\mathcal{L}/\partial x_{k_2},\\
c_k^{(2,I)} &= -2\,(y_1 s_k - x_1 t_k) + f_k\,(G^R y_1 - G^I x_1) \quad \to\ \partial\mathcal{L}/\partial y_{k_2}.
\end{aligned} \tag{22}$$

These are, line for line, the eight statements in the loop of `accumulate_gradient` in `c_funcs.c`.

2.5 The phase prior and the gain scale

With $\phi_a = \mathrm{atan2}(y_a, x_a)$, we have $\partial\phi_a/\partial x_a = -y_a/|g_a|^2$ and $\partial\phi_a/\partial y_a = x_a/|g_a|^2$, so

$$\frac{\partial\mathcal{L}_\phi}{\partial(x_a, y_a)} = \kappa\,\Phi\,\frac{(-y_a,\ x_a)}{|g_a|^2}, \qquad \kappa = \frac{2}{N_{\mathrm{ant}}^2\sigma_\phi^2}, \quad \Phi = \sum_a \phi_a. \tag{23}$$

The optimizer works with scaled gains $\tilde{g} = s\,g$. The kernels receive $g = \tilde{g}/s$ and multiply the result by $1/s$.

2.6 The complete gradient

Collecting everything:

$$\boxed{\frac{\partial\mathcal{L}}{\partial(x_a, y_a)} = \frac{1}{s}\left[\sum_{k:\,k_1=a}\left(c_k^{(1,R)}, c_k^{(1,I)}\right) + \sum_{k:\,k_2=a}\left(c_k^{(2,R)}, c_k^{(2,I)}\right) + \kappa\,\Phi\,\frac{(-y_a, x_a)}{|g_a|^2}\right]} \tag{24}$$

The rest of these notes describes how the kernels evaluate the right-hand side. Three things are needed:

1. the vector $\boldsymbol{q}$, which is the hard part (Sec. 3);
2. per-baseline arithmetic giving (22) (kernel K2);
3. for each antenna, the sum over the baselines it belongs to (kernel K3).

3 The vector q: two projections and a precision trap

3.1 Structure

Expand $\mathbf{S}$ in (13) and use $\boldsymbol{\Delta}^T\mathbf{G}^T = \boldsymbol{\Delta}_g^T$:

$$\boldsymbol{q} = \boldsymbol{\Delta}\left(\boldsymbol{\Delta}_g^T\boldsymbol{p}\right) + \boldsymbol{\Sigma}\left(\boldsymbol{\Sigma}_g^T\boldsymbol{p}\right). \tag{25}$$

Define the two projections

$$\boxed{\boldsymbol{\delta}_b = \boldsymbol{\Delta}_{g,b}^T\,\mathbf{C}^{-1}\boldsymbol{d} \in \mathbb{R}^{n_{\mathrm{eig}}}\ \text{(one per group)}, \qquad \boldsymbol{\sigma} = \boldsymbol{\Sigma}_g^T\,\mathbf{C}^{-1}\boldsymbol{d} \in \mathbb{R}^{n_{\mathrm{src}}}\ \text{(global)}.} \tag{26}$$

Row r of $\boldsymbol{q}$, in group $b(r)$, is then

$$q_r = \sum_e \Delta_{re}\, \delta_{b(r),e} + \sum_c \Sigma_{rc}\, \sigma_c. \tag{27}$$

The gain rotation $\mathbf{G}^T$ never has to be applied explicitly: it is absorbed into the gained matrices $\boldsymbol{\Delta}_g, \boldsymbol{\Sigma}_g$, which are already in memory. The projections use the *gained* factors. The expansion (27) uses the *ungained* ones.

3.2 What the projections mean

Read the model generatively. The data are $\boldsymbol{d} = \boldsymbol{\Sigma}_g \boldsymbol{a} + \boldsymbol{\Delta}_g \boldsymbol{c} + \boldsymbol{n}$, with unit-variance Gaussian amplitudes $\boldsymbol{a} \sim \mathcal{N}(0, \mathbf{I})$ for the sources, $\boldsymbol{c} \sim \mathcal{N}(0, \mathbf{I})$ for the diffuse modes, and noise $\boldsymbol{n} \sim \mathcal{N}(0, \mathbf{N})$. For a linear Gaussian model $\boldsymbol{d} = \mathbf{A}\boldsymbol{x} + \boldsymbol{n}$ with prior $\boldsymbol{x} \sim \mathcal{N}(0, \mathbf{I})$, the posterior mean is $\hat{\boldsymbol{x}} = \mathbf{A}^T(\mathbf{A}\mathbf{A}^T + \mathbf{N})^{-1}\boldsymbol{d}$. With $\mathbf{A} = [\boldsymbol{\Sigma}_g,\ \boldsymbol{\Delta}_g]$ this is

$$(\boldsymbol{\sigma},\ \boldsymbol{\delta}) = (\hat{\boldsymbol{a}},\ \hat{\boldsymbol{c}})\ : \tag{28}$$

the two projections are the Wiener-filtered estimates of the source and diffuse amplitudes. By (27), $\boldsymbol{q} = \boldsymbol{\Sigma}\hat{\boldsymbol{a}} + \boldsymbol{\Delta}\hat{\boldsymbol{c}}$ is the ungained sky signal those amplitudes predict.

3.3 Why the direct sum fails in float32

CPU CorrCal computes $\boldsymbol{\sigma}$ the obvious way, as $\sum_r (\boldsymbol{\Sigma}_g)_{rc}\, p_r$ over all n_{row} rows. Consider the size of the answer. A bright source seen on every baseline at high signal-to-noise has a large $\boldsymbol{\Lambda} = \boldsymbol{\Sigma}_g^T \mathbf{M}^{-1} \boldsymbol{\Sigma}_g$, and the estimate it produces, $\hat{\boldsymbol{a}} = (\mathbf{I} + \boldsymbol{\Lambda})^{-1} \boldsymbol{\Sigma}_g^T \mathbf{M}^{-1} \boldsymbol{d}$, is order unity. Each term $(\boldsymbol{\Sigma}_g)_{rc}\, p_r$ of the direct sum carries a factor N_r^{-1}, while the sum itself has been shrunk by $(\mathbf{I} + \boldsymbol{\Lambda})^{-1}$. The terms therefore cancel down to a result many orders of magnitude smaller than the terms themselves.

On the accuracy-test grid (18×32 antennas, $n_{\text{row}} = 331\,200$, $\min N = 1.75 \times 10^{-6}$), the ratio $\sum_r |(\boldsymbol{\Sigma}_g)_{rc}\, p_r| \,/\, |\sigma_c|$ is 1.6×10^6. In float32, $\boldsymbol{p}$ itself is accurate to about 7×10^{-6}. Multiplying a 10^{-5} input error by a 10^6 cancellation leaves no correct digits. Measured with $\boldsymbol{p}$ in float32 and the sum taken *exactly*, $\boldsymbol{\sigma}$ is off by 300%. In float64 the same cancellation still costs six digits but leaves about ten, which is why the CPU code never noticed. The per-group projection $\boldsymbol{\delta}_b$ has the same problem in its most constrained groups, where the cancellation reaches 2.6×10^6.

Precision of the accumulator cannot fix this. The lost information is in $\boldsymbol{p}$, not in the summation.

3.4 Computing the projections without cancellation

The fix is to apply the shrinkage explicitly, through the small triangular factors, rather than letting it emerge from cancellation.

Identity (source projection)**.** *Let* $\boldsymbol{u} = \boldsymbol{\Sigma}_g^T \mathbf{M}^{-1} \boldsymbol{d}$. *Then*

$$\boldsymbol{\sigma} = \boldsymbol{\Sigma}_g^T \mathbf{C}^{-1} \boldsymbol{d} = (\mathbf{I} + \boldsymbol{\Lambda})^{-1} \boldsymbol{u} = \mathbf{L}_s^{-T} \mathbf{L}_s^{-1} \boldsymbol{u}. \tag{29}$$

Proof. By Woodbury, $\mathbf{C}^{-1} = \mathbf{M}^{-1} - \mathbf{M}^{-1}\boldsymbol{\Sigma}_g (\mathbf{I} + \boldsymbol{\Lambda})^{-1} \boldsymbol{\Sigma}_g^T \mathbf{M}^{-1}$. Multiply on the left by $\boldsymbol{\Sigma}_g^T$ and on the right by $\boldsymbol{d}$: $\boldsymbol{\sigma} = \boldsymbol{u} - \boldsymbol{\Lambda}(\mathbf{I} + \boldsymbol{\Lambda})^{-1} \boldsymbol{u} = (\mathbf{I} + \boldsymbol{\Lambda})^{-1} \boldsymbol{u}$. □

The vector $\boldsymbol{u}$ itself is cheap. From $\mathbf{M}^{-1} = \mathbf{N}^{-1} - \bar{\boldsymbol{\Delta}}\bar{\boldsymbol{\Delta}}^T$ and $\mathbf{B}_b = \boldsymbol{\Sigma}_{g,b}^T \bar{\boldsymbol{\Delta}}_b$,

$$\boldsymbol{u} = \sum_b \Big[\underbrace{\boldsymbol{\Sigma}_{g,b}^T \mathbf{N}_b^{-1} \boldsymbol{d}_b}_{\text{row reduction}} - \mathbf{B}_b \underbrace{\big(\bar{\boldsymbol{\Delta}}_b^T \boldsymbol{d}_b\big)}_{\text{row reduction}} \Big]. \tag{30}$$

Neither reduction involves $\boldsymbol{p}$. The large factor N^{-1} multiplies the raw data, and the result is *large*, not a small residue. The 10^6 shrinkage happens afterwards, in one multiplication by a well-conditioned $n_{\text{src}} \times n_{\text{src}}$ matrix.

Identity (diffuse projection)**.** *Let $\boldsymbol{r} = \boldsymbol{d} - \boldsymbol{\Sigma}_g \boldsymbol{\sigma}$, the data with the estimated sources subtracted. Then*

$$\boldsymbol{\delta}_b = (\mathbf{I} + \mathbf{T}_b)^{-1} \boldsymbol{\Delta}_{g,b}^T \mathbf{N}_b^{-1} \boldsymbol{r}_b = \mathbf{R}_b \mathbf{R}_b^T \boldsymbol{y}_b, \qquad \boldsymbol{y}_b = \boldsymbol{\Delta}_{g,b}^T \mathbf{N}_b^{-1} \boldsymbol{r}_b. \tag{31}$$

Proof. From the Woodbury form above, $\mathbf{C}^{-1}\boldsymbol{d} = \mathbf{M}^{-1}\big(\boldsymbol{d} - \boldsymbol{\Sigma}_g(\mathbf{I} + \boldsymbol{\Lambda})^{-1}\boldsymbol{u}\big) = \mathbf{M}^{-1}\boldsymbol{r}$. On block b, the first-level Woodbury identity gives $\boldsymbol{\Delta}_{g,b}^T \mathbf{M}_b^{-1} = (\mathbf{I} - \mathbf{T}_b(\mathbf{I} + \mathbf{T}_b)^{-1})\boldsymbol{\Delta}_{g,b}^T \mathbf{N}_b^{-1} = (\mathbf{I} + \mathbf{T}_b)^{-1}\boldsymbol{\Delta}_{g,b}^T \mathbf{N}_b^{-1}$. Finally, $(\mathbf{I} + \mathbf{T}_b)^{-1} = \mathbf{L}_b^{-T}\mathbf{L}_b^{-1} = \mathbf{R}_b\mathbf{R}_b^T$. □

Read as statistics, (29) and (31) are Gaussian elimination in a natural order. First estimate the source amplitudes against the diffuse-plus-noise covariance $\mathbf{M}$. Subtract them. Then estimate each group's diffuse amplitudes from the source-subtracted residual. In float32 emulation on the test grid, the gradient error falls from 2.9×10^{-2} (direct sums) to 4.9×10^{-6} (this formulation).

4 The kernels

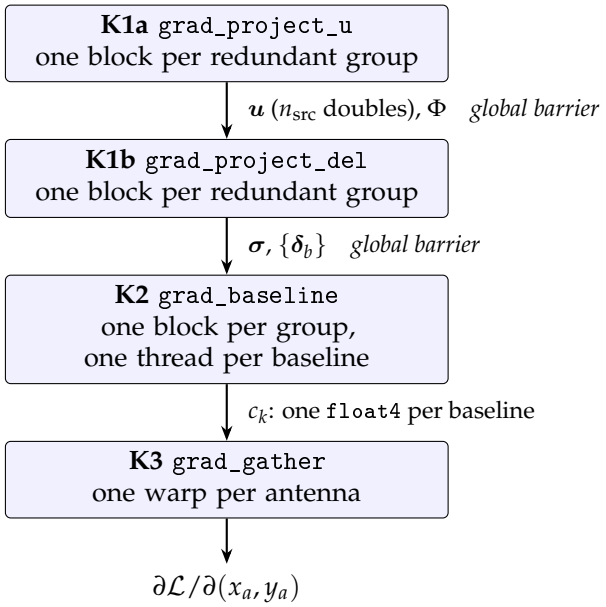


Figure 1: The four kernels, launched in order on one stream. Arrow labels are the only intermediate results written to global memory. The kernel boundaries sit exactly at the global reductions: $\boldsymbol{u}$ sums over all rows, so every block must finish K1a before any block can form $\boldsymbol{\sigma}$ in K1b.

One pattern throughout. The three group-level kernels reuse the reduction pattern of the matvec kernel (`sparse_cov_reduce`) and of `cap_reduce`. CUDA block b owns redundant group b. Each thread starts at row $e_b + \text{tid}$, advances 128 rows at a time, and accumulates into private registers. A five-step `__shfl_down_sync` tree then sums the 32 lanes of each warp. Lane 0 of each warp adds the warp total into shared memory, and the block result is read from shared memory after a barrier. Because the warp-level step needs no shared memory and no atomics, the only atomics are one per warp into shared memory and one per block into global memory.

4.1 K1a: the source estimate's right-hand side

Computes: $\boldsymbol{u}$ of (30), and the phase sum Φ.

1. *Row loop.* Each thread accumulates two partial vectors over its rows of group b:

$$\xi_e \mathrel{+}= \bar{\Delta}_{re}\, d_r \quad (e < n_{\text{eig}}), \qquad \mu_c \mathrel{+}= (\boldsymbol{\Sigma}_g)_{rc}\, \frac{d_r}{N_r} \quad (c < n_{\text{src}}). \tag{32}$$

2. *Block reduction.* Warp shuffles and then shared memory give $\boldsymbol{\xi}_b = \bar{\boldsymbol{\Delta}}_b^T \boldsymbol{d}_b$ and $\boldsymbol{\mu}_b = \boldsymbol{\Sigma}_{g,b}^T \mathbf{N}_b^{-1} \boldsymbol{d}_b$.

3. *Group contribution.* Thread $c < n_{\text{src}}$ forms $(\boldsymbol{\mu}_b - \mathbf{B}_b \boldsymbol{\xi}_b)_c$ in double and adds it into the global u_c with a double-precision `atomicAdd`. After all blocks finish, $\boldsymbol{u} = \sum_b (\boldsymbol{\mu}_b - \mathbf{B}_b \boldsymbol{\xi}_b)$, which is (30).

4. *Phase sum* (only when $\sigma_\phi < \infty$). A grid-stride loop over antennas sums $\operatorname{atan2}(y_a, x_a)$, reduced by warp shuffle and one atomic per warp into Φ.

K1a: math ↔ code

$\boldsymbol{\xi}_b \leftrightarrow$ `xd`, $\boldsymbol{\mu}_b \leftrightarrow$ `us`, $\mathbf{B}_b \leftrightarrow$ `B[b]`, $\boldsymbol{u} \leftrightarrow$ `u_acc` (zeroed by the launcher), $\Phi \leftrightarrow$ `phs_sum`.

4.2 K1b: the two Wiener estimates

Computes: $\boldsymbol{\sigma}$ of (29) and every $\boldsymbol{\delta}_b$ of (31).

1. *Source estimate.* Thread 0 of every block solves the two triangular systems in double:

$$\boldsymbol{v} = \mathbf{L}_s^{-1} \boldsymbol{u}, \qquad \boldsymbol{\sigma} = \mathbf{L}_s^{-T} \boldsymbol{v}, \tag{33}$$

   i.e. $v_i = \sum_{j \le i} (\mathbf{L}_s^{-1})_{ij} u_j$ and $\sigma_i = \sum_{j \ge i} (\mathbf{L}_s^{-1})_{ji} v_j$. The result goes to shared memory. This costs about n_{src}^2 operations, so every block repeats it, which avoids an extra launch. Block 0 also writes $\boldsymbol{\sigma}$ to global memory for K2.

2. *Row loop.* Each thread forms the source-subtracted, noise-weighted residual on its rows and projects it onto the gained diffuse modes:

$$\frac{r_r}{N_r} = \frac{d_r - \sum_c (\boldsymbol{\Sigma}_g)_{rc}\, \sigma_c}{N_r}, \qquad y_e \mathrel{+}= (\boldsymbol{\Delta}_g)_{re}\, \frac{r_r}{N_r}. \tag{34}$$

3. *Block reduction* to $\boldsymbol{y}_b = \boldsymbol{\Delta}_{g,b}^T \mathbf{N}_b^{-1} \boldsymbol{r}_b$.

4. *Diffuse estimate.* Thread 0 applies $\mathbf{R}_b\mathbf{R}_b^T$, exploiting that $\mathbf{R}_b$ is upper triangular:

$$z_i = \sum_{j \le i} R_{b,ji}\, y_j, \qquad \delta_{b,i} = \sum_{j \ge i} R_{b,ij}\, z_j, \tag{35}$$

and writes $\boldsymbol{\delta}_b$ to global memory.

K1b: math ↔ code

$\boldsymbol{\sigma}$ ↔ `w` (shared) and `sig_tmp` (global), $\boldsymbol{y}_b$ ↔ `y`, `y_red`, $\mathbf{R}_b$ ↔ `R_del + b*N_EIG*N_EIG`, $\boldsymbol{z}$ ↔ `z`, $\boldsymbol{\delta}_b$ ↔ `del_tmp[b]`.

4.3 K2: everything that happens on one baseline

Computes: the four contributions (22) of every baseline.

The block for group b first copies $\boldsymbol{\delta}_b$ and $\boldsymbol{\sigma}$ into shared memory. Its threads then stride over the *baselines* k of the group, from $e_b/2$ to $e_{b+1}/2$, not over rows. One thread handles both rows $2k, 2k+1$, because s_k and t_k couple the real and imaginary parts. Entirely in registers, thread k evaluates:

$$q_k^{R,I} = \sum_e \Delta_{2k(+1),e}\, \delta_{b,e} + \sum_c \Sigma_{2k(+1),c}\, \sigma_c \qquad \text{eq. (27)} \tag{36}$$

$$\bar{P}_k = \textstyle\sum_e (\bar{\Delta}^2_{2k,e} + \bar{\Delta}^2_{2k+1,e}) + \sum_c (\bar{\Sigma}^2_{2k,c} + \bar{\Sigma}^2_{2k+1,c}) \qquad \text{eq. (20)} \tag{37}$$

$$s_k,\ t_k = p_k^R q_k^R + p_k^I q_k^I,\ \ p_k^I q_k^R - p_k^R q_k^I \qquad \text{eq. (14)} \tag{38}$$

$$G_k^R,\ G_k^I \text{ from the four gains of antennas } k_1, k_2 \qquad \text{eq. (2)} \tag{39}$$

$$f_k = 2\, N_{2k}\, \bar{P}_k / |G_k|^2 \qquad \text{eq. (19)} \tag{40}$$

and stores $(c_k^{(1,R)}, c_k^{(1,I)}, c_k^{(2,R)}, c_k^{(2,I)})$ as one 16-byte `float4`.

Every auxiliary vector of the CPU implementation ($\mathbf{G}^T\boldsymbol{p}$, $\boldsymbol{q}$, s, t, `inv_power`) exists only in registers. None of them is written to memory. This is the fusion the design set out to achieve. The only per-baseline output is c_k.

K2: math ↔ code

q_k^R, q_k^I ↔ `q0, q1`, $\bar{P}_k$ ↔ `pbar`, s_k, t_k ↔ `S, T`, $G_k^{R,I}$ ↔ `Gr, Gi`, f_k ↔ `pf`, c_k ↔ `contrib[k]` = `(c.x, c.y, c.z, c.w)`.

4.4 K3: from baselines to antennas

Computes: the sums over baselines in (24), plus the prior and the scale.

The scatter problem. Equation (24) sums over the baselines that *contain* antenna a. The CPU loop runs over baselines and scatters each c_k into two antennas' slots. Run in parallel over baselines, that scatter is a write race, because every antenna sits on $N_{\text{ant}} - 1$ baselines handled by different threads.

Why warp-aggregated atomics do not help. Within one redundant group, each antenna appears at most once as k_1 and at most once as k_2. The 32 lanes of a K2 warp therefore target 32 *different* antennas, and a shuffle has nothing to combine. On a 5×6 grid with baselines in redundant order, 0% of lanes in a warp share a target. Every contribution would reach global memory as its own atomic, and floating-point atomics would make the gradient depend on scheduling order from one call to the next.

The gather. Instead, invert the map once per solve. Number the $2N_{\mathrm{bl}}$ contributions $\epsilon = 2k + \rho$, where $\rho = 0$ if the antenna is k_1 and $\rho = 1$ if it is k_2. Viewed as an array of (Re, Im) pairs, `contrib` holds the pair for entry ϵ at position ϵ. Sort the entries by owning antenna into a compressed-sparse-row structure:

$$\mathcal{E}_a = \{\epsilon : \text{entry } \epsilon \text{ belongs to antenna } a\} = \texttt{csr_idx}\big[\texttt{csr_ptr}[a] : \texttt{csr_ptr}[a+1]\big]. \tag{41}$$

Warp a then computes

$$\frac{\partial \mathcal{L}}{\partial (x_a, y_a)} = \frac{1}{s}\Big[\sum_{\epsilon \in \mathcal{E}_a} c_\epsilon \; + \; \kappa\, \Phi\, \frac{(-y_a, x_a)}{|g_a|^2} \Big], \tag{42}$$

with lanes striding through $\mathcal{E}_a$, accumulating in double, and combining with `__shfl_down_sync`. Lane 0 adds the prior term, applies $1/s$ and writes the two outputs. Each warp writes only its own antenna's two numbers, so there are no atomics and the result is bitwise reproducible. Here the shuffle reduction combines lanes that genuinely share a target. The price is one write and one read of 16 bytes per baseline.

K3: math $\leftrightarrow$ code

$\mathcal{E}_a \leftrightarrow$ `csr_idx[csr_ptr[a]:csr_ptr[a+1]]` (built once in `GradWorkspace`), $c_\epsilon \leftrightarrow$ `contrib2[`ϵ`]`, $\kappa \leftrightarrow$ `phs_coef`, $1/s \leftrightarrow$ `inv_scale`.

5 Summary

Kernel	Mathematics	Reads per row	Writes
K1a	$\boldsymbol{u} = \sum_b (\mathbf{\Sigma}_{g,b}^T \mathbf{N}_b^{-1} \boldsymbol{d}_b - \mathbf{B}_b \bar{\mathbf{\Delta}}_b^T \boldsymbol{d}_b),\ \Phi$	$\mathbf{\Sigma}_g,\ \bar{\mathbf{\Delta}},\ N,\ d$	$\boldsymbol{u}, \Phi$
K1b	$\boldsymbol{\sigma} = (\mathbf{I} + \mathbf{\Lambda})^{-1}\boldsymbol{u};\ \boldsymbol{\delta}_b = (\mathbf{I} + \mathbf{T}_b)^{-1} \mathbf{\Delta}_{g,b}^T \mathbf{N}_b^{-1} (\boldsymbol{d} - \mathbf{\Sigma}_g \boldsymbol{\sigma})_b$	$\mathbf{\Delta}_g,\ \mathbf{\Sigma}_g,\ N,\ d$	$\boldsymbol{\sigma}, \boldsymbol{\delta}_b$
K2	$\boldsymbol{q}, \bar{P}, s, t, G_k, f_k \to c_k$	$\mathbf{\Delta},\ \mathbf{\Sigma},\ \bar{\mathbf{\Delta}},\ \bar{\mathbf{\Sigma}},\ N,\ p$	c_k
K3	$\sum_{\epsilon \in \mathcal{E}_a} c_\epsilon$ + prior, $\times 1/s$	(16 B per baseline)	$\partial \mathcal{L}$

The gradient is $\mathrm{Tr}(\mathbf{C}^{-1}\partial\mathbf{C}) - \boldsymbol{p}^T \partial\mathbf{C}\, \boldsymbol{p}$ (plus the prior), evaluated baseline by baseline and gathered antenna by antenna. Everything is local to a row or a baseline except the two amplitude estimates. Those are the only steps that need a reduction across rows, and they are where the kernel boundaries fall. The amplitude estimates are also the only numerically delicate step. Computed as Wiener filters, by building a large, cancellation-free right-hand side and then shrinking it with the small Cholesky factors the inverse already produced, they survive float32. Computed as a direct inner product with $\boldsymbol{p}$, they do not.